AI-Assisted Architecture Decision Narrative

1. Starting problem

The side project selected a financial combination rather than generic CRUD:

```
Digital Wallet + Double-entry Ledger + Fraud Detection + External Banking Integration
```

The goal was to exercise real backend concerns such as concurrency, idempotency, provider failures, accounting, session security, auditability and reconciliation.

2. Why a modular monolith?

Wallet, Ledger and Transaction participate in the same atomic money movement. Splitting them into microservices immediately would introduce distributed-consistency costs too early. Domain modules remain logically separated and deployment boundaries can be reconsidered later.

3. Why MSSQL as the financial source of truth?

The system needs ACID transactions, constraints, FKs, unique keys, explicit locking and deterministic reconciliation queries. Redis is useful for fast transient state but is not allowed to decide whether money exists.

4. Why authentication first?

Wallet, BankAccount and Transfer ownership require stable customer/session identity. Registration -> OTP -> password -> login -> JWT -> refresh/session therefore came before money endpoints.

5. Why JWT plus server-side sessions?

JWT authenticates requests but does not independently provide immediate revoke. A minimal sid claim plus durable CustomerSession provides revoke/device/session checks for high-risk flows.

6. Why separate Wallet and BankAccount?

Wallet is internal customer liability/balance state. BankAccount is an external-provider relationship. Their lifecycle and source of truth differ, so they are not one aggregate.

7. Why FakeBank as a separate API?

An HTTP boundary forces the project to handle actual integration concerns: timeout, 5xx, pending state, provider-generated identity, idempotency and polling. FinWallet never accesses provider storage directly.

8. Why durable state before bank HTTP?

Bank-account opening persists internal BankAccount(Opening), closes SQL work, then calls the provider. The durable internal ID generates a deterministic provider request key so lost responses do not create duplicate external accounts.

9. Why Ledger before the transfer endpoint?

Balance answers “how much exists now?” Ledger answers “why does that balance exist?”. Money endpoints were not exposed before accounting invariants existed.

10. Why FinancialTransaction separate from LedgerJournal?

FinancialTransaction represents the business operation/lifecycle; LedgerJournal represents the accounting effect. Keeping them separate supports failure, reversal, idempotency resource identity and reconciliation.

11. Why an explicit transfer store?

SqlWalletTransferPostingStore owns idempotency, both wallet updates, FinancialTransaction and Ledger in one MSSQL transaction. Locking and commit order are correctness rules here, so explicit SQL is preferred over a generic repository abstraction.

12. Why durable idempotency in MSSQL?

Mobile timeout, Gateway timeout, duplicate clicks and retries can resend the same financial command. Final duplicate protection is not delegated to process cache or Redis. The identity uses Scope + CustomerId + Key plus a canonical payload hash.

13. Why fraud before the money SQL transaction?

External HTTP is slow and uncertain. Fraud is evaluated first; a short atomic financial SQL transaction starts only after final Allow. Required fraud unavailable => fail closed.

14. Why completed replay before fraud?

A transaction completed yesterday should not be denied today when the client simply retries the same request under changed fraud rules. Completed immutable results are replayed first.

15. Why controller-based APIs?

All HTTP services use controllers for consistent enterprise Web API conventions, attributes, Swagger and explicit action contracts. Minimal API conventions are not mixed into the same codebase.

16. Why is ServiceResult only an HTTP contract?

ServiceResult<T> standardizes client responses. Making Domain/Application depend on it would unnecessarily couple business objects to HTTP transport.

17. Why YARP after financial core?

Routing infrastructure cannot fix incorrect money logic. Gateway was added as the edge/platform boundary after auth, persistence, ledger and transfer correctness were established.

18. Why service-to-service through Gateway?

Provider adapters use /providers/*. Two credentials exist:

```
FinWallet -> Gateway   InternalServiceKey
Gateway -> Backend     DownstreamServiceKey
```

This makes Gateway an enforceable trust boundary rather than only a client convention.

19. Why Shared.Web?

Duplicating Swagger/rate/CORS/header code across seven Web APIs would cause configuration drift. FinWallet.Shared.Web centralizes hosting concerns without becoming a business framework.

20. Why are some values configurable and others not?

Addresses, timeouts, pools, rate limits, body/header/connection limits, CORS, Swagger and YARP destinations are operational and therefore configurable. HS256 algorithm, PBKDF2 V1 migration semantics, Debit=Credit, financial decimals and durable idempotency semantics remain correctness/security invariants rather than arbitrary runtime toggles.

21. Why not expand Redis into a general cache?

Current use focuses on OTP/transient state. Wallet balances and final idempotency truth remain in MSSQL to avoid unnecessary invalidation and correctness risks.

22. Why add mocks but not treat them as sufficient?

xUnit + Moq can verify Application orchestration calls. SQL Serializable/range-lock behavior and YARP route correctness require real infrastructure tests.

23. Implementation sequence

```
scope/invariants
-> architecture boundaries
-> registration/auth/session
-> communication/fraud providers
-> wallet/ledger domain
-> FakeBank + BankAccount
-> persistence
-> wallet APIs
-> financial transaction/idempotency schema
-> atomic transfer store
-> fraud/session transfer orchestration
-> YARP Gateway
-> Shared.Web + Swagger/security
-> config/performance tuning
-> xUnit/Moq + CI tests
-> bilingual/current documentation
```

24. Natural next order

1. BankDeposit;
2. BankWithdrawal + cutoff;
3. integration/concurrency tests;
4. Outbox/Inbox + notification;
5. FraudEvents/manual review;
6. transaction history;
7. refund/reversal;
8. reconciliation;
9. OpenTelemetry/masked logging/operations.

25. Core rule

```
Do not expose the easy endpoint before the hard invariant underneath it exists.
```

Transfer is not exposed before ledger/idempotency; bank flows should not be exposed before durable state; provider retries should not exist without idempotency.